

Unit Test Report 401

FPGA WHT Compression Pipeline – iverilog Testbench Results

Field	Value
Document	UTR401
Date	2026-04-25
Author	ColeZenk
Simulator	iverilog + vvp
Related SDDs	SDD400, SDD401

Scope

This report covers the four testbenches written for the FPGA WHT compression pipeline. All testbenches are in `fpga/sim/`. Tests were run manually via iverilog from that directory.

TB ID	File	DUT(s)	Result
TB001	<code>wht8_tb.v</code>	<code>wht8.v</code>	PASS
TB002	<code>coeff_tb.v</code>	<code>coeff.v</code> , <code>seq_msk.v</code>	PASS
TB003	<code>esp_interface_tx_tb.v</code>	<code>esp_interface.v</code>	FAIL
TB004	<code>blk_schd_tb.v</code>	<code>blk_schd.v</code> (+ sub)	TIMEOUT

TB001 – `wht8_tb`

Objective

Verify the 8-point combinational WHT butterfly (`wht8.v`) against known vectors. `wht8.v` is the core arithmetic primitive used in all 16 passes of the 2D WHT inside `blk_schd.v`. If this is wrong, every downstream coefficient is wrong.

Thought Process

The WHT butterfly has no state – it is purely combinational. The expected outputs are computable by hand or by the Python reference (`sim/wht_core.py` FWHT class). The testbench strategy was to cover edge cases: all-zero input (no excitation), DC impulse (uniform), alternating ± 1 (Hadamard row 7), unit impulse at position 0, linear ramp, and a mix of positive and negative values. If all six pass, the butterfly is correct for both sign handling and magnitude, and there is no need to enumerate all 256^8 inputs.

Setup

Parameter	Value
W (bits)	16 (signed)
Inputs	8 x signed 16-bit
Outputs	8 x signed 16-bit
Latency	0 (combinational)

Results

Test	Input description	Result
1	All zeros	PASS
2	DC ones [1,1,...,1]	PASS
3	Alternating +1/-1	PASS
4	Impulse at x[0]=64	PASS
5	Ramp [0,1,2,...,7]	PASS
6	Mixed negatives	PASS

Result: 6/6 PASS. No errors.

Observations

Butterfly is correct. The output scaling is consistent with unnormalized WHT (sum, not average). This matches sim/wht_core.py. No issues found.

TB002 – coeff_tb

Objective

Verify the Top-K coefficient extractor (coeff.v) including the sequency mask behavior embedded in it (seq_thresh=6, DC skip). The extractor must: skip DC (r=0,c=0), skip positions where r+c>6, apply magnitude threshold (thresh=400), quantize (q_shift=3), and return up to TOP_K=5 results ranked by magnitude.

Thought Process

coeff.v is a sequential FSM (IDLE -> SCAN -> DONE). The critical paths to test are: threshold behavior (just below, exactly at, just above), DC skip, sequency mask boundary (r+c=6 should pass, r+c=7 should be masked), Top-K replacement (more than 5 qualifying coefficients forces replacement of the current minimum slot), and sign handling (negative coefficients must be quantized with correct sign). A zero-input frame (no nonzero coefficients) is also important because blk_schd checks coeff_n==0 to skip EMIT.

Setup

Parameter	Value
W	16
TOP_K	5
thresh	400
q_shift	3
seq_thresh	6

Results

Test	Description	Result
1	All zeros – no coefficients	PASS
2	Single AC coeff above threshold	PASS
3	DC term skipped	PASS
4	Sequency mask (r+c>6 zeroed)	PASS
5	Top-K replacement (>5 above thr)	PASS
6	Negative coefficient sign	PASS

Result: 6/6 PASS. No errors.

Observations

Extractor is correct. DC is always skipped. Sequency mask matches seq_msk.v behavior (r+c>6 excluded). Top-K replacement correctly evicts the smallest-magnitude slot. Sign of quantized output is preserved. No issues found.

TB003 – esp_interface_tx_tb

Objective

Verify the MISO TX path of esp_interface.v. The test drives a known 16-byte pattern into tx_data (advancing on tx_next) and clocks the SPI master interface, capturing MISO byte by byte. All captured bytes must match the expected tx_data sequence.

This testbench was written specifically to hunt for Bug 1: suspected inter-byte skew causing the ESP32 to read corrupted coefficient bytes from the FPGA.

Thought Process

The MISO TX path in esp_interface.v preloads tx_sr from tx_data when: a) CS asserts (initial preload), or b) byte_done_d fires (one cycle after the 8th rising SCLK edge).

After preload, tx_sr shifts left on each falling SCLK edge, and MISO = tx_sr[7].

The concern: between two back-to-back bytes, byte_done fires on the 8th rising edge. byte_done_d fires one cycle later (the registered version). tx_sr is reloaded from tx_data. But immediately following byte_done_d, the master drives another falling SCLK edge (the start of the next byte's first bit). That falling edge hits tx_sr BEFORE the master has sampled tx_sr[7]. The shift happens, eating the MSB of the freshly-loaded byte.

If this bug is present, every byte[N>=1] arrives at the master as the correct byte shifted left by 1 (bit 0 forced to 0, original MSB lost).

To make the test definitive: tx_mem was loaded with values that clearly distinguish a 1-bit-left-shifted version from the original (e.g. 0xDE shifted left = 0xBC, not 0xDE).

Setup

Signal	Configuration
CLK_PERIOD	37 ns (27 MHz system clock)
SPI_HALF	222 ns (CLK_PERIOD * 6)
tx_mem	16 known bytes (0xDE, 0xAD, 0xBE, ...)
tx_data	Driven from testbench register file
tx_next	Advances tx_data on each byte boundary

Three tests: - Test 1: 8-byte sequential readback - Test 2: CS deassert/reassert (pointer reset behavior) - Test 3: 16 back-to-back bytes with no inter-byte gap

Results

Test	Description	Bytes checked	Errors
1	8-byte sequential readback	8	7
2	CS toggle re-preload	1	1
3	16 bytes, no inter-byte gap	16	14

Total errors: 22. FAIL.

Byte 0 (first byte of each CS assertion) passes. All subsequent bytes fail. Captured byte pattern: got[N] = expected[N] « 1 (MSB=0, lower 7 bits shifted up).

Root Cause – Bug 1

The between-bytes falling SCLK edge arrives in the same system clock cycle that `byte_done_d` is asserted (or the cycle immediately after). The `tx_sr` reload fires and then the shift fires, consuming `tx_sr[7]` before the master can sample it on the following rising edge. The master samples the new byte starting from bit 6 (one position early), which is equivalent to receiving the correct value shifted left by 1.

The first byte in each CS assertion passes because there is no prior between-bytes edge – the CS assert preload fires and SCLK starts fresh from 0. The bug only manifests on the transition between byte N and byte N+1.

Fix direction: do not shift `tx_sr` on the falling edge that immediately follows a reload. A reliable gate: `if (sclk_fall & cs_active & bit_count != 0)` – `bit_count` is 0 on the first falling edge after preload because the rising edge (which increments `bit_count` to 0 mod 8) has not yet occurred.

TB004 – blk_schd_tb

Objective

Verify the block scheduler (`blk_schd.v`) end-to-end: - All-zero frame produces zero TX output (no coefficients emitted)
- A frame with a non-zero 8x8 block at (0,0) with a gradient pattern produces TX output

The testbench mocks the PSRAM with a local array and a 1-cycle busy model.

Thought Process

`blk_schd.v` is the most complex module in the design. Its LOAD state drives `psram_read` high, sets `psram_addr`, then waits for `psram_busy` to deassert before consuming `psram_rdata`. `psram_pending` tracks whether a read has been issued but not yet consumed. The handshake is:

1. `psram_pending=0, psram_busy=0` -> issue read (`psram_read=1, psram_pending=1`)
2. Wait for `psram_busy` to assert (mock latency)
3. Wait for `psram_busy` to deassert -> consume `rdata`, `psram_pending=0`

The testbench mock uses:

```
if (psram_read) psram_busy <= 1; else if (psram_busy && psram_read_d) begin psram_rdata <= data; psram_busy <= 0; end
```

The intent is: assert busy when read fires, deassert one cycle later when `psram_read` has gone low and `psram_read_d` is high (edge detection). But the race is:

Cycle A: `psram_read=1, psram_pending=1` -> mock: `psram_busy` goes 1
Cycle B: `psram_read=0` (cleared at top of always block), `psram_busy=1` -> mock waits
Cycle B: `blk_schd` sees `psram_busy=1` (via `psram_pending` check) -> stays in LOAD, waits
Cycle C: `psram_read_d=1, psram_busy=1` -> mock delivers `rdata`, clears `psram_busy`

This should work for the first read. But for back-to-back reads in the same block: after `psram_pending` is cleared (first read consumed), `blk_schd` immediately issues a second read (`psram_read=1` again). If this happens in the same cycle that the mock is clearing `psram_busy` (cycle C above), the mock sees `psram_read=1`, sets `psram_busy=1` again. But it also clears `psram_busy` in the else-if branch. The two assignments to `psram_busy` in the same always block mean the last assignment wins – which in a sequential always block resolves to the last procedural statement. The result depends on statement ordering and can leave `psram_busy` stuck at 1.

The simulation hangs at the timeout (50,000,000 ns) without reaching `frame_done`.

Results

Test	Description	Result
1	All-zero frame (<code>cur==prev</code>)	TIMEOUT
2	Non-zero block (gradient)	NOT RUN

Result: TIMEOUT. No test completed. Simulation killed at 50 ms timeout.

Root Cause – Bug 2 (Testbench)

The PSRAM mock uses implicit edge detection (`psram_read_d + psram_busy`) to sequence its state transitions. This is fragile when `blk_schd` issues a second read in the same cycle that the mock is completing the first. `psram_busy` can remain stuck asserted permanently, stalling `blk_schd` in the LOAD state indefinitely.

This is a testbench bug, not an RTL bug. `blk_schd.v`'s psram handshake is consistent with the interface contract. The mock does not faithfully model a fixed-latency memory.

Fix direction: replace the mock with an explicit N-cycle countdown register:

- On `psram_read`: latch `psram_addr`, start countdown = N
- Each cycle: decrement countdown
- When countdown reaches 0: drive `psram_rdata`, clear `psram_busy`

This is unconditional and does not race with the DUT's read pulses.

Collateral – Block Scheduler EMIT Analysis

While debugging TB004, the EMIT state termination condition was reviewed:

`emit_byte` (5 bits) is compared against: $4'd1 + \{1'b0, \text{coeff_n}\} * 3$

For `coeff_n=5` (TOP_K): $1 + 15 = 16$. `emit_byte` is 5-bit; 16 fits (no overflow). Total bytes output: 17 (2 header + 5 coefficients x 3 bytes each). Correct.

The `emit_sub` / `emit_idx` interlock was also traced: `emit_idx` advances on `emit_sub=2` (concurrent with `q_lo` output). The next cycle, `emit_sub=0` with the new `emit_idx`, selecting the correct `rc_byte` for the next coefficient. No timing issue found here.

Summary

TB	DUT(s)	Tests	Pass	Fail	Notes
TB001	wht8.v	6	6	0	Core butterfly correct
TB002	coeff.v, seq_msk.v	6	6	0	Top-K extractor correct
TB003	esp_interface.v	24	2	22	Bug 1: MISO inter-byte shift
TB004	blk_schd.v (+sub)	2	0	0	TIMEOUT – Bug 2: mock deadlock

Open Items

ID	Module	Description	Priority
BUG-1	esp_interface.v	MISO inter-byte shift (all bytes[N>=1])	HIGH
BUG-2	blk_schd_tb.v	PSRAM mock deadlock, testbench fix needed	HIGH
BUG-3	top.v tx_buf	Registered BRAM read lag on tx_rd_data	MEDIUM
BUG-4	cam_spi.c (ESP32)	Heap leak from DMA buffer drop -> crash	HIGH

BUG-1 must be fixed before any coefficient data from the FPGA can be trusted. BUG-2 must be fixed before TB004 can validate `blk_schd.v` pipeline correctness. BUG-3 timing analysis should follow the BUG-1 fix since both affect the MISO pipeline.